

CREDIT RISK PREDICTION SYSTEM

A Supervised Machine Learning Approach to Loan Applicant Classification

Machine Learning 600 Assignment

Dataset: Credit Risk Dataset (Kaggle)

Name and Surname: _____

Student ITS No: _____

Problem Statement

A mid-sized financial services company needs to modernise credit approvals.

Existing rule-based system:

- Cannot capture nonlinear relationships between applicant variables
- Produces false approvals → financial losses
- Produces false rejections → lost business, poor customer experience

Objective

Build a supervised classification system to predict whether a loan applicant is:

LOW RISK (loan_status = 0)

HIGH RISK (loan_status = 1)

Following the full ML lifecycle: acquisition → EDA → preprocessing → PCA → modelling → evaluation → optimisation.

Dataset Overview

32,581

Applicant Records

12

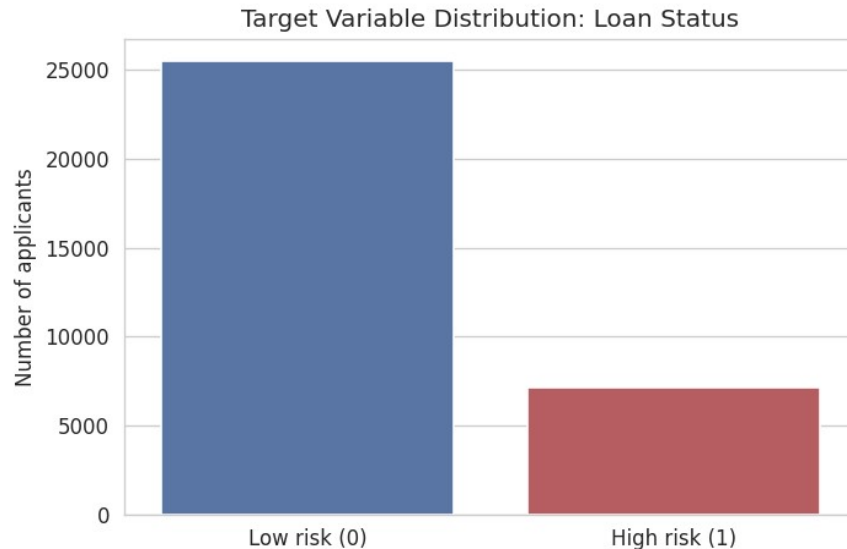
Original Features

22%

High-Risk Rate

Source: Kaggle - laotse/credit-risk-dataset

- Demographics: person_age, person_income, person_home_ownership, person_emp_length
- Loan details: loan_intent, loan_grade, loan_amnt, loan_int_rate, loan_percent_income
- Credit history: cb_person_default_on_file, cb_person_cred_hist_length
- Target: loan_status (0 = low risk, 1 = high risk)



Target variable is imbalanced (~78% / 22%) → accuracy alone is not a reliable metric; precision, recall and F1 are reported throughout.

Data Issues Identified

1

Missing values

loan_int_rate: 9.6% missing

person_emp_length: 2.7% missing

2

Invalid values

person_age up to 144 years

person_emp_length up to 123 years

3

Duplicate rows

157 exact duplicate records found

4

Outliers

person_income & loan_amnt strongly
right-skewed with high-end outliers

Preprocessing Summary

01

CLEAN

Removed 902 rows with impossible age/employment values + 157 duplicates

02

IMPUTE

loan_int_rate → median by loan_grade;
person_emp_length → overall median

03

TREAT OUTLIERS

IQR capping on person_income, loan_amnt,
person_emp_length

04

ENCODE

loan_grade → ordinal; home_ownership & loan_intent → one-hot

05

SCALE

StandardScaler fit on training data only (avoids leakage)

06

SPLIT

80% train / 20% test, stratified on loan_status

Feature scaling is essential: Logistic Regression's optimiser and KNN's distance metric are scale-sensitive, and PCA is variance-based — all three would be distorted by unscaled features.

Principal Component Analysis

Why PCA?

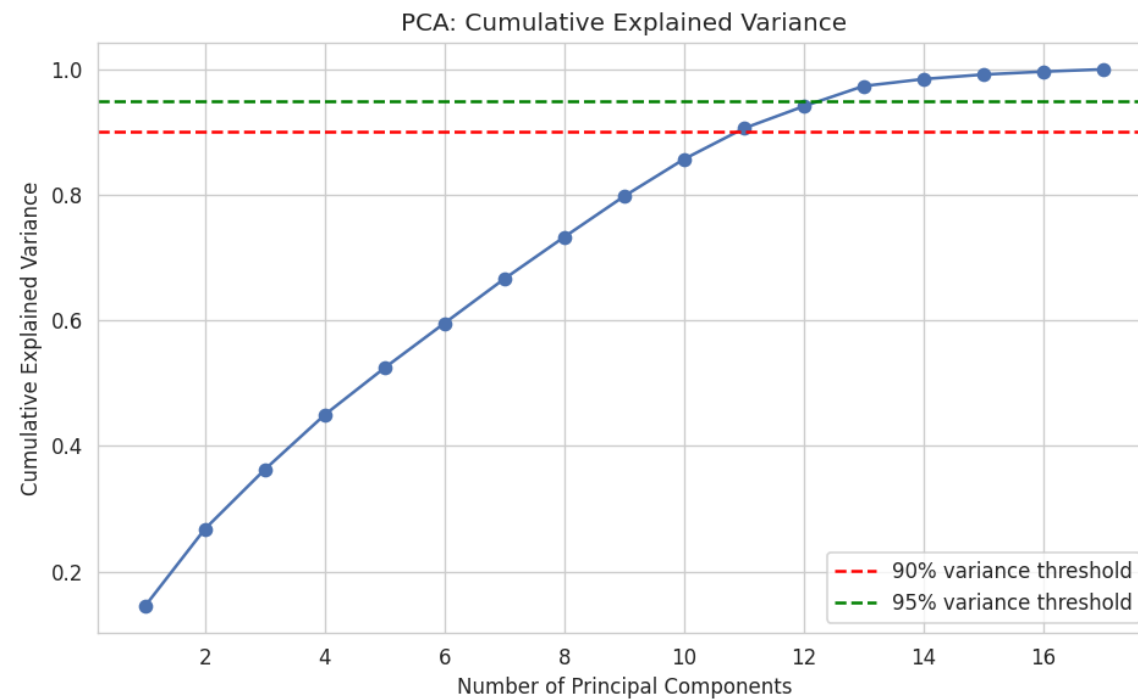
The correlation heatmap showed redundancy between features (e.g. `person_age` and `cb_person_cred_hist_length`) — PCA compresses this without losing much signal.

11

components selected
(from 17 original features)

90.6%

cumulative variance
retained ($\geq 90\%$ target)



Model Comparison — Original Features

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.8549	0.7416	0.5037	0.5999
k-Nearest Neighbours	0.8987	0.8754	0.6189	0.7252
Decision Tree	0.8983	0.7551	0.7834	0.7690

BEST MODEL

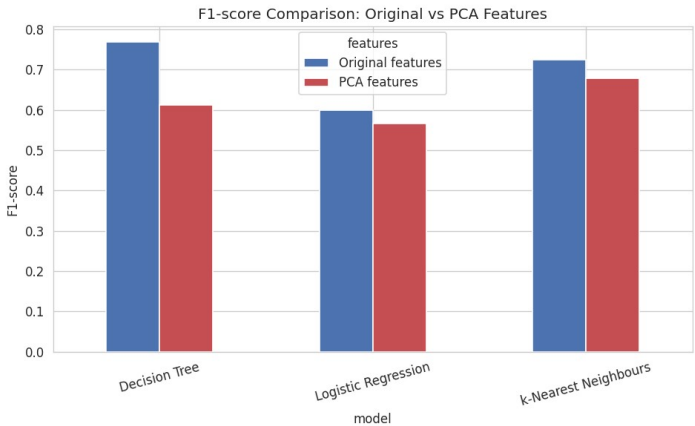
Decision Tree

Highest F1-score and recall — best balance of catching high-risk applicants without excessive false positives.

On an imbalanced target, F1-score (harmonic mean of precision & recall) is the primary model-selection metric — not accuracy.

Model Comparison — PCA Features

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.8449	0.7152	0.4684	0.5661
k-Nearest Neighbours	0.8809	0.8133	0.5822	0.6786
Decision Tree	0.8303	0.6044	0.6204	0.6123



F1: original vs PCA features (bar chart)

Every model loses a small amount of F1-score after PCA (e.g. Decision Tree: 0.769 → 0.612). This is the expected dimensionality-reduction trade-off: compressing 17 features into 11 components discards some class-discriminative detail in exchange for a smaller, less collinear feature space — useful when compute cost or multicollinearity is a greater concern than squeezing out maximum accuracy.

Hyperparameter Tuning — Decision Tree

GridSearchCV (5-fold CV, scoring = F1)

criterion: gini

max_depth: 10

min_samples_split: 10

min_samples_leaf: 1

Model	Accuracy	Precision	Recall	F1-score
Before tuning	0.8983	0.7551	0.7834	0.7690
After tuning	0.9316	0.9463	0.7247	0.8208

Bias-Variance Impact

Untuned tree: grows unrestricted → low bias, high variance → overfits training data.

Tuned tree: depth/leaf constraints trade a little bias for much lower variance → better generalisation on unseen test data.

Conclusion and Insights

1

Tuned Decision Tree is the recommended production model: $F1 = 0.821$, precision = 0.946, recall = 0.725 on held-out test data

2

PCA offers useful compression (17 $\rightarrow$ 11 features, 90.6% variance) but at a small accuracy cost — best used when interpretability/collinearity outweighs raw performance

3

False negatives (high-risk applicants misclassified as low-risk) carry greater financial cost than false positives — recall on the high-risk class should weight the final deployment threshold

4

Recommend piloting the tuned model alongside the existing rule-based system before full replacement